

.. 29 | 08 | 26 ..
o o o o o o o

LISTA 1 - Introdução à ES

1. São eles: Processo, Métodos e Ferramentas.

Processo: É a todo, sendo responsável por definir os requisitos para o desenvolvimento e manutenção do software e além disso serve também como uma estrutura de encadeamento de métodos e ferramentas.

Método: Tendo objetividade, contidas no desenvolver do processo, um método é a forma com a qual a atividade será realizada. Um exemplo seria a parte comunicativa para definição de requisitos, por exemplo, como uma parte do processo que pode ter como método a execução de entrevistas ou workshops.

Ferramentas: Entram como formas de automação para os métodos utilizados ao longo do processo, uma ferramenta bem feita ou ainda bem empregada é capaz de incluir, reduzir erros humanos durante os métodos. Importante ressaltar que a ferramenta mal empregada pode ser geradora de erros, um exemplo abstrato pode ser um martelo para fixar um parafuso, o que funcionaria mas geraria problemas indesejáveis no processo.

1. Coloque* em uma panela funda leite condensado, a margarina e o chocolate em pó. | * verbos
| são me
| todos
2. Cozinhe* [no fogo] em fogo médio e mexa* sem parar com uma colher de pau. | - fumaça
| quente
3. Cozinhe* até que o brigadeiro começa a desgrudar da panela.
4. Deixe esfriar* bem, então unte* as mãos com margarina, faça as bolinhas* e envolva-as* em chocolate granulado. |
|





2. A existência de um processo é garantida durante o desenvolvimento de um software, no entanto caso ele não esteja devidamente documentado ele é chamado de implícito. Um processo implícito depende portanto de boca a boca e entendimento do código legado, o que pode ser gerador de erros por novatos na empresa por exemplo.

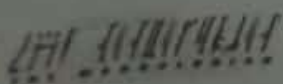
3. São processos pré-fabricados que possuem estilo próprio, que devem ser adaptados para seu uso.

Devem ser usados para seguir com alguns mandamentos da engenharia de software, como a padronização e o controle de qualidade por exemplo, tem esses processos pré-fabricados é uma forma de garantir de início de de coisas bem estabelecido.

Devem ser adaptados, uma vez que um processo genérico não é capaz de se adequar perfeitamente às necessidades do projeto que será desenvolvido ou do cliente entre outros.

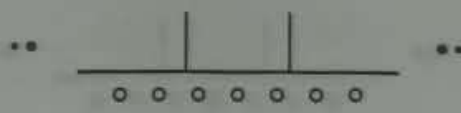
4. O modelo incremental foca na entrega de pequenas partes funcionais do software de forma sucessiva ao longo do processo com o objetivo de garantir que o sistema em desenvolvimento seja de fato o desejado pelo cliente. Importante ressaltar que no final de cada ciclo temos um software que pode ser colocado em execução.

O modelo evolutivo a ideia principal é colher feedback, portanto a cada fim de ciclo ao invés de ter um software pronto para execução temos mais informações para desenvolver no próximo ciclo e assim por diante.



Jandaia

© & TM Lucidchart Ltd.



5. Casca: O mais simples de todos, colocando em sequência os métodos do processo que só aconteçam uma vez. Exatidão para um software com requisitos estáveis e bem definidos, mas não é sempre esse o caso por isso não é tão usado.

Incremental: Como dito no item anterior ele conta seus métodos em sequência uma vez por ciclo, porém contém vários ciclos e o objetivo final de cada um deles é deixar pronto um software funcional que desempenhe um requerimento do software completo.

RAD: Também baseado no cascata o RAD dá foco para as partes mais custosas do projeto (Modelagem e Construção) fazendo com que essas sejam desenvolvidas em paralelo, tendo uma equipe por funcionalidade, por exemplo e por conta disso precisa de uma equipe grande e também uma fase de integração uma vez que o software foi desenvolvido todo separado.

Prototipação: De modo geral é usado como um ciclo de vida secundário que tem como principal objetivo ser rápido e fácil de implementar com o objetivo de trazer feedback do cliente e é preferencialmente descartável.

Espiral: Temos um exemplo de ciclo de vida evolutivo, como dito no item anterior ele tem como principal objetivo evoluir um software completo do início ao fim, o que significa que diferente do incremental que traz parte por parte do software já pronto a espiral não, ele trata de desenvolver o todo, passo por passo, ciclo a ciclo até a implantação final.



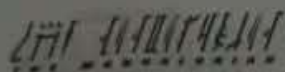


6. A escolha entre entrega por tempo ou por funcionalidades traz mudanças relevantes no design do software e até mesmo em seu processo. Quando a entrega por tempo é a escolhida há maior previsibilidade para a entrega, naturalmente ela vem mais rápida, o que possibilita um feedback do cliente de forma mais rápida, entretanto, prazos curtos demais podem causar problemas no processo por gerar maior pressão e, portanto, podendo gerar bugs no produto final. Já na entrega por funcionalidades é evidente que o software será entregue 100% funcional, entretanto é extremamente difícil saber quando ele ficará pronto e ter feedback do cliente.

7. Como explicado no item anterior, entrega por tempo e qualidade baixa podem sim estar intrinsicamente ligados, porém se o tempo combinado para a entrega estiver de acordo com o escopo do software, daí é bem possível que a entrega seja feita com qualidade mesmo que por tempo.

8. Manutenção de software é o processo de modificar um sistema de software ou componente, depois de entregue, para corrigir falhas, melhorar desempenho ou outros atributos, ou adaptar a mudanças no ambiente. IEEE Std 620.12-1990

Pensando no estilo de vida em cascata, a manutenção é feita após a entrega do software como um todo, o que é simples e direto de se pensar, diferente do em cascata o incremental permite que exista manutenção pedida pelo usuário logo após sua primeira entrega, uma vez que parte do software já está em uso.





2. Correções.

Emergencial: Não programada, portanto é uma correção que deve ser feita de forma emergencial (Lapa bugado) para que o software volte a funcionar o mais rápido possível, porém apenas essa solução pode ser possível de mais problemas futuros, portanto é inteligente começar uma manutenção corretiva logo em seguida.

Corretivo: Ela, assim como a anterior, também é reativa mas não é crítica e portanto um típico bug que não vai tirar o sistema do ar mas que naturalmente trará acesso a alguma usabilidade do software.

Preventiva: Agora não temos uma manutenção reativa, a ideia aqui é que a equipe de desenvolvimento prevê um bug futuro e o corrige antes que aconteça. O melhor exemplo é o bug do milênio, softwares que usassem comparações de anos para calcular idades por exemplo, ficariam inutilizáveis, uma vez teriam tirado o ano de nascimento do indivíduo do "atual" (isso representado como 00) tornando o valor da idade do mesmo negativo.

Evolução:

Adaptativa: Quando o software deve se adequar a uma mudança no ambiente, um exemplo pode ser por exemplo uma mudança de imposto promulgada em Brasília que tornaria as relações de um software de contabilidade falsas, portanto deve haver uma adaptação no software.

Perfeccionista: Nesse caso o usuário é protagonista do pedido de mudança, esse tipo de manutenção é geralmente o pedido de melhorias de velocidade de resposta do software ou as melhorias de atributos de qualidade do software.



.. 29 | 08 | 26 ..
o o o o o o o o

LISTA 2

1. A principal diferença entre os dois paradigmas é a forma com a qual o programa é composto, no caso do procedural por funções e rotinas que manipulam os dados de forma separada, já os OO unem objetos e métodos numa mesma entidade, portanto em OO um objeto pode realizar um método, diferente do procedural que tem funções que alteram dados. É o comum entre os dois é o objetivo de modularizar a lógica de um programa quebrando problemas complexos em menores.

2. As classes em OO são moldes para objetos, de forma simples é a que um objeto obrigatoriamente contém. Pensando no sistema de uma universidade no qual é necessário que um aluno tenha: Nome e matrícula; a classe vai seguir esse padrão obrigando que cada instância (objeto) dele, contenha esses dados para que métodos funcionem devidamente.

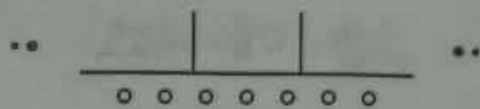
3. Encapsulamento tem como principal objetivo a segurança. Definindo um método como público ele pode ser usado em qualquer lugar do pacote o que pode gerar problemas de inconsistência. Imagine por exemplo que temos o método de mudar a coordenada de um ponto de uma figura geométrica como público, é crucial que ele exista para a existência de outros métodos como de alteração de comprimento, por exemplo, porém caso usado de forma isolada esse método pode violar as características físicas da figura, portanto é crucial que esse método seja privado.

11111111111111111111



Jandaia

© & TM Acornfilm Ltd.



4. A herança em OO é uma excelente forma de organização e coerência de código. Quando uma classe filha herda métodos de uma classe mãe ela garante que seus objetos funcionem com os da mãe e também da classe filha, a diferença nesse caso é não precisar reescrever métodos e garantir que toda instância da classe filha seja pr. toda uma da classe mãe. Já o polimorfismo é quando objetos de classes diferentes respondem à mesma chamada de método, cada um de uma forma, o que permite tratar objetos distintos de maneira unificada e genérica.

5. Uma classe abstrata é uma classe que não pode ter seus métodos instanciados, ela serve de molde para classes que são filhas da mesma obrigando as elas que instanciem esses métodos. Já as interfaces agem como contratos de comportamento, portanto é basicamente uma obrigação de como a classe deve funcionar tendo que conter determinado método por exemplo.

.. 29 | 08 | 26 ..
o o o o o o o o

LISTA - 3

1. A análise OO tem como objetivo entender o que precisa ser feito para resolver o problema no campo de investigação, e isso é feito levantando requisitos para o sistema, já o projeto está no campo de solução e é a como resolver o problema.

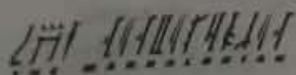
2. Modelos são observações de realidade, que tem como objetivo fazer um tratado de aspectos reais de forma que mostrem o que deve ser feito no desenvolvimento. Eles servem para simplificar a realidade e garantir que a equipe se comunique com uma abstração do que o software será no futuro para garantir que todos entendam por completo o software.

3. O modelo age como um tipo de conjunto de diagramas de maneira a priorizar o todo desde que a empresa compacte toda a informação nele. Mesmo sendo um diagrama parte de um modelo, ele tem como objetivo mostrar um ponto de vista do modelo por exemplo casos de uso.

4. Casos de uso: É o diagrama mais próximo de análise, sendo composto pelos papéis (⌘) e casos de uso (○), o diagrama relaciona ambos, portanto quem pode fazer cada coisa.

Classes: Também em nível de análise, ele tem todas as classes (□) do sistema tendo como objetivo dar uma visão geral de alto nível de todo o sistema.

Transição de estados: Demonstra todos os estados possíveis de um objeto complexo, tem como objetivo mostrar claramente suas transições.



.. 31 | 08 | 26 ..
o o o o o o o o

LISTA - 4

1. A Engenharia de Requisitos é uma fase bem estruturada de traduzir as necessidades do mundo real de um negócio em um design técnico de software. Tanto como principal objetivo fornecer uma documentação sistemática e controlada para compreender a pedido do cliente, avaliar a viabilidade, negociar uma solução, especificar funcionalidades e gerenciar mudanças.

As 7 tarefas da engenharia de requisitos:

1. Compreensão:

Estabelecer o entendimento inicial do problema. É o momento em que as equipes se reúnem para identificar quem são os envolvidos, qual é a natureza da solução desejada e quais são os benefícios de negócio que justificam o projeto.

2. Elicitação (levantamento):

Fazer parte ou extrair diretamente as necessidades do cliente. No capítulo esse é uma tarefa dita complexa graças a problemas de escopo (fronteiras do sistema mal definidas), de entendimento (clientes não sabem exatamente o que querem) e de validade (requisitos mudando durante o desenvolvimento).

3. Elaboração: As informações isoladas obtidas no levantamento são refinadas e organizadas para criar um modelo de análise técnica. A equipe desenvolve cenários detalhados para visualizar o sistema em funcionamento.

4. Negociação: Como os clientes frequentemente pedem mais do que o orçamento do projeto permite, os desenvolvedores e os interessados discutem para priorizar recursos. O objetivo é alcançar um acordo realista sobre o escopo final.



.. 31 | 08 | 26 ..
o o o o o o o

5. Especificação: A consolidação do que será construído, servindo como o documento principal para a fase de design. Pode assumir formatos variados, desde um documento formal e modelos gráficos até um protótipo executável.

6. Validação: Os produtos de engenharia gerados passam por revisões técnicas formais para garantir qualidade. Basicamente por erros de lógica, ambiguidades ou omissões antes que os programadores comecem a escrever código.

7. Gerenciamento: Atividades contínuas de rastreamento para controlar as inevitáveis alterações de requisitos, permitindo avaliar o impacto no tempo e no custo sempre que uma mudança é pedida.

Identificação dos interessados e seus pontos de vista:

Durante a concepção, é vital mapear os atores do sistema (gerentes, usuários finais, suporte técnico...). Como diferentes interessados possuem perspectivas e necessidades conflitantes, reconhecer essas divergências antecipadamente garante que o sistema não seja projetado de forma enviesada para favorecer apenas um departamento.

Desenvolvimento de casos de uso:

Utilizados extensivamente na fase de elaboração, os casos de uso capturam a visão do usuário sobre as funcionalidades. Eles descrevem cenários específicos de interação entre um ator e o sistema, detalhando o fluxo de eventos passo a passo sem se preocupar com os detalhes de implementação do código.

Elementos do Modelo de Análise: Elementos baseados em cenário e classe, componentes e orientados a fluxo.

Negociação e Validação:

Exige a colaboração do cliente para a classificação de suas necessidades por importância, fora a da equipe com o custo. E logo após o esboço ser documentado, a validação age como filtro final de qualidade. Através de checagem a equipe testa e inicia o processo.



.. 30 | 08 | 26 ..
o o o o o o o

LISTA-5

1. Um caso de uso é uma técnica utilizada para capturar requisitos funcionais, sendo o agrupamento de cenários de uso com objetivo em comum. O mesmo descreve diferentes possíveis momentos durante uma ação. Por exemplo durante a ação de comprar, existe o cenário típico onde a compra vai do início ao fim da forma que se espera e também os cenários alternativos, extensões, que podem ocorrer em alguns momentos durante a ação.

2. Como dito brevemente no item anterior, não. Um caso de uso foca no cenário típico que é também o que mais ocorre e que é o sucesso! Mas nem sempre o ator consegue ir do início ao fim do caso dentro do cenário típico, algum problema pode ocorrer e, então deve ser mapeado como um caminho alternativo e implementado para ser executado no momento correto.

3. Os atores são entidades que usam o sistema ou melhor acessam a mesma para alguma ação, importante ressaltar que nem sempre um ator é um humano pode ser um outro sistema como o de pagamento do Visa no controle de vendas de uma farmácia por exemplo.

4. Na estrutura simples tem os elementos: Nome, cenário típico e cenários alternativos.

Na estrutura detalhada tem os elementos: Nome, lista de atores, visão geral (descrição), referências cruzadas, gatilho, pré-condições, pós-condições, cenário típico e cenários alternativos.



.. 30 | 08 | 26 ..
o o o o o o o

5.1 Um diagrama de casos de uso é composto por: atores, casos de uso, limites do sistema e relacionamentos.

6.1 Na UML as relações «extende» e «inclui» denotam relações entre casos de uso, sendo a «extende» um detalhamento que vem, justamente de um caso de uso alternativo, já a «inclui» é uma relação mais obrigatória na sequência daquela caso de uso, portanto é como se fossem o mesmo caso de uso separados por questão de coerência dos nomes dos mesmos.

